What is Python?

- Python is a general-purpose, high-level, interpreted, object-oriented programming language used for various applications.
- Python was developed by Guido Van Rossum in 1989 while working at National Research Institute in the Netherlands. But officially, Python was made available to the public in 1991.
- Python code syntax uses English keywords, making it easy to understand. Therefore, Python is recommended as the first programming language for beginners.
- In addition, it includes high-level data structures, dynamic typing, dynamic binding, and many more features that make it very attractive for rapid application development.
- Python is simple and easy to learn. Syntax emphasizes readability and therefore reduces the cost of program maintenance. In addition, it supports modules and packages, which encourages program modularity and code reuse.
- The Python interpreter and the extensive standard library are available in source or binary form for all major platforms. In addition, it has a wide range of standard and third-party libraries that helps in rapid application development.

Applications of Python

We can use Python everywhere because Python is known for its general-purpose nature, which makes it applicable in almost every domain of software development. The most common application areas are:

- Desktop Applications
- Web Applications
- Database Applications
- Network Programming
- Games
- Data Analysis Applications
- Machine Learning
- Artificial Intelligence Applications
- IoT

Install Python

It may be possible that some PCs and Macs will have Python already installed. You can check which version of Python is installed before proceeding to the installation.

Open the command line or terminal and type the below command.

```
python -version
```

- Installing or updating Python on your computer is the first step to programming in Python. There are multiple installation methods, such as installing Python using an installer or a source code (.zip file)

- Download the latest version of Python from python.org. Once you've downloaded the installer as per the operating system, Next run an installer by double-clicking on the downloaded file and following the steps.

- After installation is completed, we will get a **setup successfully** install message.

Create and Run Your First Python Program

We can run Python by using the following three ways

- Run Python using IDLE
- Run Python interactively using the command line in immediate mode
- Executing Python File

We will see each one by one but before that, let's see how to write your first Python program.
Let's write a simple statement in Python to print the 'hello world' on a screen.

- Use the `print()` function and write a message in its opening and closing brackets shown below.
- A message is a string that is a sequence of characters. In Python, strings are enclosed inside single quotes, double quotes, or triple quotes.

Run Python Using IDLE

- IDLE is an integrated development environment (IDE) for Python. The Python installer contains the IDLE module by default. Thus, when you install Python, IDLE gets installed automatically.

- Go to launchpad (for mac) and start icon (for Windows) and type IDLE, to open it. IDLE is an interactive Python Shell where you can write python commands and get the output instantly.

IDLE has features like coding hinting, syntax highlighting, checking, etc.

We can create a new file, write Python code, and save it with the `.py` extension. The `.py` is the python file extension which denotes this is the Python script.

Let's see how to create a Python script using IDLE.

- Go to the File Menu and select the new file option
- Type the same code (hello world message) in it
- Next, Go to the File menu to save it as `hello.py`

Run Python on Command Line

We can also run Python on the command line.

- Type `python` command on the command line or terminal to run Python interactively. It will invoke the interpreter in immediate mode.
- Next, type Python code and press enter to get the output.
- To exit this mode, type `quit()` and press enter.

Syntax and Indentation in Python

Syntax is the structure of language or a set of rules that defines how a Python program will be written and interpreted.

- A line containing only white space, possibly with a comment or within a code, is known as a blank line, and Python ignores it.
- In Python end of the line terminate the statement. So you don't need to write any symbol to mark the end of the line to indicate the statement termination. For example, in other programming languages like Java and C, the statement must end with a semicolon (`;`).
- In Python, we can add **multiple statements on a single line** separated by semicolons.

Indentation

- Python indentation tells a Python interpreter that the group of statements belongs to a particular block of code. The indentation makes the code look neat, clean, and more readable.
- A block is a combination of all multiple statements. Inside a code block, we group multiple statements for a specific purpose.
- In other programming languages like C or Java, use curly braces `{ }` to define a block of code. Python uses indentation to denote the code block.
- **Whitespace is used for indentation** in Python to define the indentation level. Ideally, we should use **4 spaces** per indentation level. In Python, indented code blocks are always preceded by a colon (`:`) on the previous line.

Python Statements

A **statement is an instruction that a Python interpreter can execute**. So, in simple words, we can say anything written in Python is a statement.

- Python statement ends with the token NEWLINE character. It means each line in a Python script is a statement.

- For example, `a = 10` is an assignment statement. where `a` is a variable name and 10 is its value. There are other kinds of statements such as `if` statement, `for` statement, `while` statement, etc., we will learn them in the following lessons.

Multi-Line Statements

- Python statement ends with the token NEWLINE character. But we can extend the statement over multiple lines using line continuation character (`\`). This is known as an explicit continuation.
- We can use parentheses `()` to write a multi-line statement. We can add a line continuation statement inside it. Whatever we add inside a parentheses `()` will treat as a single statement even it is placed on multiple lines.
- We can use square brackets `[]` to create a list. Then, if required, we can place each list item on a single line for better readability.
- Same as square brackets, we can use the curly `{ }` to create a dictionary with every key-value pair on a new line for better readability.

```
# list of strings
names = ['Emma',
         'Kelly',
         'Jessa']
print(names)
# dictionary name as a key and mark as a value
# string:int
students = {'Emma': 70,
            'Kelly': 65,
            'Jessa': 75}
print(students)
```

The `pass` statement

- `pass` is a null operation. Nothing happens when it executes. It is useful as a placeholder when a statement is required syntactically, but no code needs to be executed.

- For example, you created a function for future releases, so you don't want to write a code now. In such cases, we can use a `pass` statement.

```
# create a function
def fun1(arg):
    pass  # a function that does nothing (yet)
```

The `del` statement

The Python `del` statement is used to delete objects/variables.

```
del target_list
```

The `target_list` contains the variable to delete separated by a comma. Once the variable is deleted, we can't access it.

The `return` statement

- We create a function in Python to perform a specific task. The function can return a value that is nothing but an output of function execution.

- Using a `return` statement, we can return a value from a function when called.

```
# Define a function
# function acceptts two numbers and return their sum
def addition(num1, num2):
    return num1 + num2  # return the sum of two numbers
# result is the return value
result = addition(10, 20)
print(result)
```

The `import` statement

- The import statement is used to import modules. We can also import individual classes from a module.
- Python has a huge list of built-in modules which we can use in our code. For example, we can use the built-in module DateTime to work with date and time.

```
import datetime
# get current datetime
now = datetime.datetime.now()
print(now)
```

The Continue & Break Statement

- `break` Statement: The break statement is used inside the loop to exit out of the loop.

- `continue` Statement: The continue statement skip the current iteration and move to the next iteration.

Python Keywords

Python keywords are **reserved words that have a special meaning** associated with them and can't be used for anything but those specific purposes. Each keyword is designed to achieve specific functionality.
Python keywords are **case-sensitive**.

- All keywords contain only letters (no special symbols)
- Except for three keywords (`True`, `False`, `None`), all keywords have lower case letters.

As of Python 3.9.6, there are **36 keywords** available. This number can vary slightly over time. We can use the following two ways to get the list of keywords in Python.

- **keyword module**: The keyword is the buil-in module to get the list of keywords. Also, this module allows a Python program to determine if a string is a keyword.
- **`help()` function**: Apart from a keyword module, we can use the `help()` function to get the list of keywords.

```
import keyword
print(keyword.kwlist)
help("keywords")
```

```
Here is a list of the Python keywords.  Enter any keyword to get more help.

False               break               for                 not
None                class               from                or
True                continue            global              pass
__peg_parser__      def                 if                  raise
and                 del                 import              return
as                  elif                in                  try
assert              else                is                  while
async               except              lambda              with
await               finally             nonlocal            yield
```

Types of Keywords

- **Value Keywords:** True, False, None.
- **Operator Keywords:** and, or, not, in, is
- **Conditional Keywords:** if, elif, else
- **Iterative and Transfer Keywords:** for, while, break, continue, else
- **Structure Keywords:** def, class, with, as, pass, lambda
- **Import Keywords:** import, from, as
- **Returning Keywords:** return, yield
- **Exception-Handling Keywords:** try, except, raise, finally, else, assert
- **Variable Handling Keywords:** del, global, nonlocal
- **Asynchronous Programming Keywords:** async, await

Python Variables

- A **variable is a reserved memory area (memory address) to store value**. For example, we want to store an employee's salary. In such a case, we can create a variable and store salary using it. Using that variable name, you can read or modify the salary amount.
- In other words, a variable is a value that varies according to the condition or input pass to the program. Everything in Python is treated as an object so every variable is nothing but an object in Python.
- A variable can be either **mutable** or **immutable**. If the variable's value can change, the object is called mutable, while if the value cannot change, the object is called immutable.
- We can assign a value to the variable at that time variable is created. We can use the assignment operator `=` to assign a value to a variable.
- The operand, which is on the left side of the assignment operator, is a variable name. And the operand, which is the right side of the assignment operator, is the variable's value.

```
<code>variable_name = variable_value</code>
```

Integer variable

The `int` is a data type that returns integer type values (signed integers); they are also called **ints** or **integers**. The integer value can be positive or negative without a decimal point.

```
# create integer variable
age = 28
print(age)  # 28
print(type(age))  # <class 'int'>
```

Float variable

Floats are the values with the **decimal point** dividing the integer and the fractional parts of the number. Use float data type to store decimal values.

```
# create float variable
salary = 10800.55
print(salary)  # 10800.55
print(type(salary))  # <class 'float'>
```

Complex type

The `complex` is the numbers that come with the real and imaginary part. A complex number is in the form of a+bj, where a and b contain integers or floating-point values.

```
a = 3 + 5j
print(a)  # (3+5j)
print(type(a))  # <class 'complex'>
```

String variable

In Python, a string is a **set of characters** represented in quotation marks. Python allows us to define a string in either pair of **single** or **double** quotation marks. For example, to store a person's name we can use a string type.

Rules and naming convention for variables and constants

A name in a Python program is called an identifier. An identifier can be a variable name, class name, function name, and module name.

- The name of the variable and constant should have a combination of letters, digits, and underscore symbols.
- Alphabet/letters i.e., lowercase (a to z) or uppercase (A to Z)
- Digits(0 to 9)
- Underscore symbol (_)
- The variable name and constant name should make sense.
- Don't' use special symbols in a variable name.
- For declaring variable and constant, we cannot use special symbols like $, #, @, %, !~, etc. If we try to declare names with a special symbol, Python generates an error.
- Variable and constant should not start with digit letters.
- Identifiers are case sensitive.
- To declare constant should use capital letters.
- Use an underscore symbol for separating the words in a variable name.

We can assign a single value to multiple variables simultaneously using the assignment operator =.

```
a = b = c = 10
print(a) # 10
print(b) # 10
print(c) # 10
```

Variable scope

- **Scope**: The scope of a variable refers to the places where we can access a variable.

- Depending on the scope, the variable can categorize into two types **local variable** and the **global variable.**

Local variable

A local variable is a variable that is accessible inside a block of code only where it is declared. That means, If we declare a variable inside a method, the scope of the local variable is limited to the method only. So it is not accessible from outside of the method. If we try to access it, we will get an error.

```
def test1():  # defining 1st function
    price = 900  # local variable
    print("Value of price in test1 function :", price)
def test2():  # defining 2nd function
    # NameError: name 'price' is not defined
    print("Value price in test2 function:", price)
# call functions
test1()
test2()
```

Global variable

A Global variable is a variable that is defined outside of the method (block of code). That is accessible anywhere in the code file.

```
price = 900  # Global variable
def test1():  # defining 1st function
    print("price in 1st function :", price)  # 900
def test2():  # defining 2nd function
    print("price in 2nd function :", price)  # 900
# call functions
test1()
test2()
```

Python Operators

Operators are special symbols that perform specific operations on one or more operands (values) and then return a result. For example, you can calculate the sum of two numbers using an addition (`+`) operator.

Python has seven types of operators that we can use to perform different operation and produce a result.

- Arithmetic operator
- Relational operators
- Assignment operators
- Logical operators
- Membership operators
- Identity operators
- Bitwise operators

Arithmetic operator

Arithmetic operators are the most commonly used. The Python programming language provides arithmetic operators that perform addition, subtraction, multiplication, and division. It works the same as basic mathematics.

There are seven arithmetic operators we can use to perform different mathematical operations, such as:

- `+` (Addition)
- `-` (Subtraction)
- `*` (Multiplication)
- `/` (Division)
- `//` Floor division)
- `%` (Modulus)
- `**` (Exponentiation)

Relational (comparison) operators

- Relational operators are also called comparison operators. It performs a comparison between two values.
- It returns a boolean True or False depending upon the result of the comparison.
- Python has the following six relational operators. Assume variable `x` holds 10 and variable `y` holds 5.

Operator	Description	Example
> (Greater than)	It returns True if the left operand is greater than the right	`x > y` result is `True`
< (Less than)	It returns True if the left operand is less than the right	`x < y` result is `False`
== (Equal to)	It returns True if both operands are equal	`x == y` result is `False`
!= (Not equal to)	It returns True if both operands are equal	`x != y` result is `True`
>= (Greater than or equal to)	It returns True if the left operand is greater than or equal to the right	`x >= y` result is `True`
<= (Less than or equal to)	It returns True if the left operand is less than or equal to the right	`x <= y` result is `False`

Assignment operators

- In Python, Assignment operators are used to assigning value to the variable.
- Assign operator is denoted by = symbol. For example, `name = "Jessa"` here, we have assigned the string literal 'Jessa' to a variable name.
- Also, there are shorthand assignment operators in Python. For example, `a+=2` which is equivalent to `a = a+2`.

Operator	Meaning	Equivalent
= (Assign)	a=5Assign 5 to variable a	a = 5
+= (Add and assign)	a+=5Add 5 to a and assign it as a new value to a	a = a+5
-= (Subtract and assign)	a-=5Subtract 5 from variable a and assign it as a new value to a	a = a-5
*= (Multiply and assign)	a*=5Multiply variable a by 5 and assign it as a new value to a	a = a*5
/= (Divide and assign)	a/=5Divide variable a by 5 and assign a new value to a	a = a/5
%= (Modulus and assign)	a%=5Performs modulus on two values and assigns it as a new value to a	a = a%5
**= (Exponentiation and assign)	a**=5Multiply a five times and assigns the result to a	a = a**5
//= (Floor-divide and assign	a//=5Floor-divide a by 5 and assigns the result to a	a = a//5

Logical operators

Logical operators are useful when checking a condition is `true` or not. Python has three logical operators. All logical operator returns a boolean value `True` or `False` depending on the condition in which it is used.

Operator	Description	Example
`and` (Logical and)	True if both the operands are True	a and b
`or` (Logical or)	True if either of the operands is True	a or b
`not` (Logical not)	True if the operand is False	not a

Bitwise Operators

In Python, bitwise operators are used to performing bitwise operations on integers. To perform bitwise, we first need to convert integer value to binary (0 and 1) value.

The bitwise operator operates on values bit by bit, so it's called **bitwise**. It always returns the result in decimal format. Python has 6 bitwise operators listed below.

- `&` Bitwise and
- `|` Bitwise or
- `^` Bitwise xor
- `~` Bitwise 1's complement
- `<<` Bitwise left-shift
- `>>` Bitwise right-shift

Python Data Types

- Data types specify the different sizes and values that can be stored in the variable. For example, Python stores numbers, strings, and a list of values using different data types.

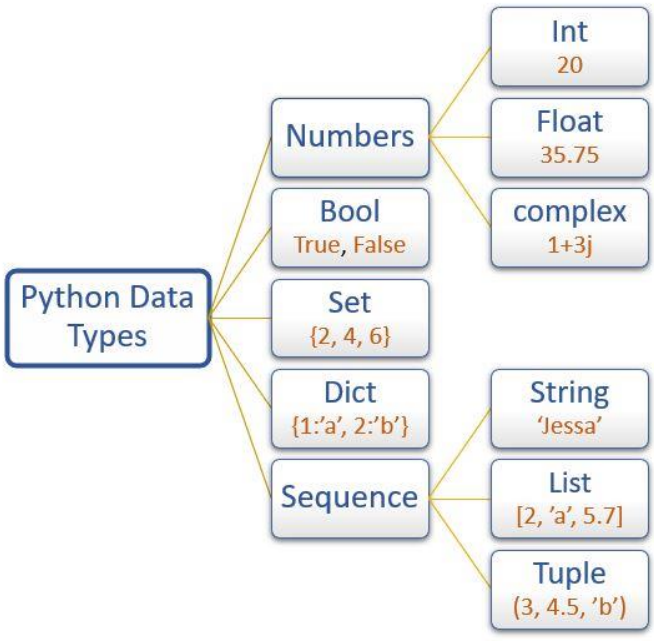
- Python is a dynamically typed language; therefore, we do not need to specify the variable's type while declaring it.

- Whatever value we assign to the variable based on that data type will be automatically assigned. For example, `name = 'Jessa'` here Python will store the name variable as a `str` data type.

- No matter what value is stored in a variable (object), a variable can be any type like int, float, str, list, set, tuple, dict, bool, etc.

There are mainly four types of basic/primitive data types available in Python

- **Numeric**: int, float, and complex
- **Sequence**: String, list, and tuple
- **Set**
- **Dictionary** (dict)



To check the data type of variable use the built-in function `type()` and `isinstance()`.

- The `type()` function returns the data type of the variable.
- The isinstance() function checks whether an object belongs to a particular class.

Data type	Description	Example
`int`	To store integer values	`n = 20`
`float`	To store decimal values	`n = 20.75`
`complex`	To store complex numbers (real and imaginary part)	`n = 10+20j`
str	To store textual/string data	`name = 'Jessa'`
bool	To store boolean values	`flag = True`
list	To store a sequence of mutable data	`l = [3, 'a', 2.5]`
tuple	To store sequence immutable data	`t =(2, 'b', 6.4)`
dict	To store key: value pair	`d = {1:'J', 2:'E'}`
set	To store unorder and unindexed values	`s = {1, 3, 5}`
frozenset	To store immutable version of the set	`f_set=frozenset({5,7})`
range	To generate a sequence of number	`numbers = range(10)`
bytes	To store bytes values	`b=bytes([5,10,15,11])`

Python Casting: Type Conversion and Type Casting

In Python, we can convert one type of variable to another type. This conversion is called type casting or type conversion.
In casting, we convert variables declared in specific data types to the different data types.
Python performs the following two types of casting.

- **Implicit casting**: The Python interpreter automatically performs an implicit Type conversion, which avoids loss of data.
- **Explicit casting**: The explicit type conversion is performed by the user using built-in functions.

To perform a type casting, we are going to use the following built-in functions

- `int()`**:** convert any type variable to the integer type.
- `float()`**:** convert any type variable to the float type.
- `complex()`**:** convert any type variable to the complex type.
- `bool()`**:** convert any type variable to the bool type.
- `str()`**:** convert any type variable to the string type.

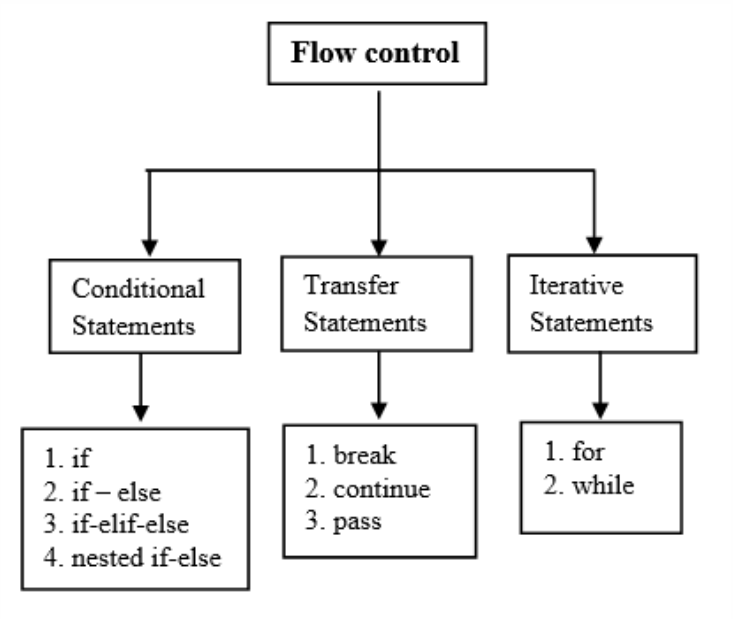
In type casting, data loss may occur because we enforce the object to a specific data type.

Python Control Flow Statements and Loops

- In Python programming, flow control is the order in which statements or blocks of code are executed at runtime based on a condition.

- The flow control statements are divided into **three categories.**

  - Conditional statements
  - Iterative statements.
  - Transfer statements



Conditional Statements

In Python, condition statements act depending on whether a given condition is true or false. You can execute different blocks of codes depending on the outcome of a condition. Condition statements always evaluate to either True or False.

If statement in Python

- In control statements, The `if` statement is the simplest form. It takes a condition and evaluates to either `True` or `False`.

- If the condition is `True`, then the True block of code will be executed, and if the condition is False, then the block of code is skipped, and The controller moves to the next line.

```
if condition:
    statement 1
    statement 2
    statement n
```

If – else statement

The `if-else` statement checks the condition and executes the `if` block of code when the condition is True, and if the condition is False, it will execute the `else` block of code.

```
if condition:
   statement 1
else:
   statement 2
```

If – elif - else statement

- In Python, the `if-elif-else` condition statement has an `elif` blocks to chain multiple conditions one after another. This is useful when you need to check multiple conditions.

- With the help of `if-elif-else` we can make a tricky decision. The `elif` statement checks multiple conditions one by one and if the condition fulfills, then executes that code.

```
if condition-1:
    statement 1
elif condition-2:
    stetement 2
elif condition-3:
    stetement 3
    ...
else:
    statement
```

Nested if-else statement

- In Python, the nested `if-else` statement is an if statement inside another `if-else` statement. It is allowed in Python to put any number of `if` statements in another `if` statement.

- Indentation is the only way to differentiate the level of nesting. The nested `if-else` is useful when we want to make a series of decisions.

```
if conditon_outer:
    if condition_inner:
        statement of inner if
    else:
        statement of inner else:
    statement ot outer if
else:
    Outer else
statement outside if block
```

Single statement suites

- Whenever we write a block of code with multiple if statements, indentation plays an important role. But sometimes, there is a situation where the block contains only a single line statement.
- Instead of writing a block after the colon, we can write a statement immediately after the colon.

```
number = 56
if number > 0: print("positive")
else: print("negative")
```

Iterative Statements

In Python, iterative statements allow us to execute a block of code repeatedly as long as the condition is True. We also call it a loop statements. Python provides us the following two loop statement to perform some actions repeatedly:

- for loop
- while loop

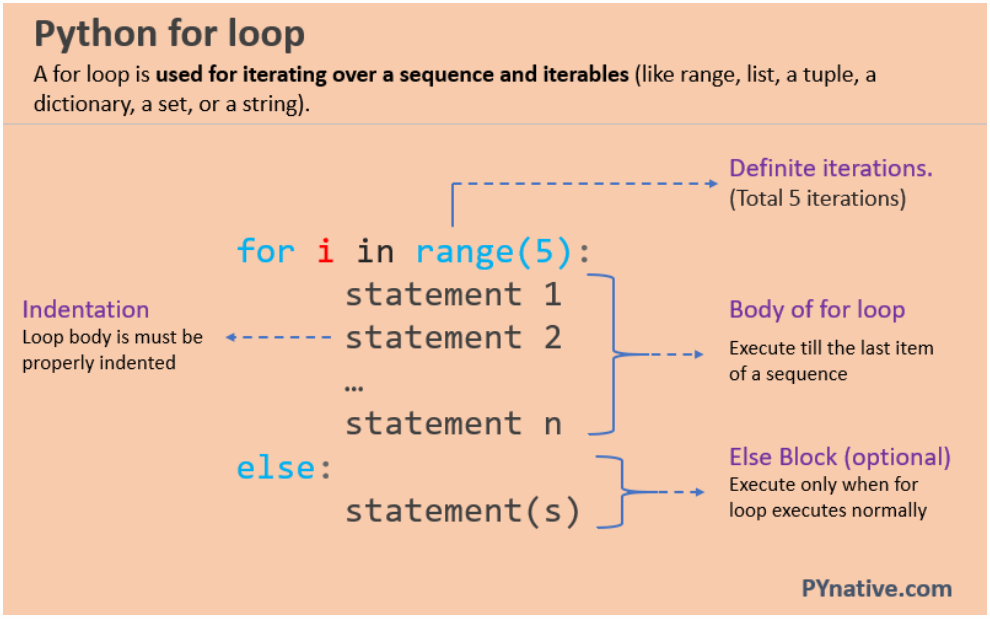
Transfer statements

In Python, transfer statements are used to alter the program's way of execution in a certain manner. For this purpose, we use three types of transfer statements.

- break statement
- continue statement
- `pass` statements

Python for Loop

- In Python, the `for` loop is used to iterate over a sequence such as a list, string, tuple, other iterable objects such as range.
- With the help of `for` loop, we can iterate over each item present in the sequence and executes the same set of operations for each item. Using a `for` loops in Python we can automate and repeat tasks in an efficient manner.




```
for i in range/sequencee:
    statement 1
    statement 2
    statement n
```

- In the syntax, `i` is the iterating variable, and the range specifies how many times the loop should run. For example, if a list contains 10 numbers then for loop will execute 10 times to print each number.
- In each iteration of the loop, the variable `i` get the current value.

for loop with range()

- The range() function returns a sequence of numbers starting from 0 (by default) if the initial limit is not specified and it increments by 1 (by default) until a final limit is reached.
- The `range()` function is used with a loop to specify the range (how many times) the code block will be executed.

If-else in for loop

- If-else is used when conditional iteration is needed.
- In this program, `for` loop statement first iterates all the elements from 0 to 20.
- Next, The `if` statement checks whether the current number is even or not. If yes, it prints it. Else, the else block gets executed.

```
for i in range(1, 11):
    if i % 2 == 0:
        print('Even Number:', i)
    else:
        print('Odd Number:', i)
```

Loop Control Statements in for loop

Loop control statements change the normal flow of execution. It is used when you want to exit a loop or skip a part of the loop based on the given condition. It also knows as transfer statements.

Break for loop

- The break statement is used to **terminate the loop**. You can use the break statement whenever you want to stop the loop. Just you need to type the break inside the loop after the statement, after which you want to break the loop.
- When the `break` statement is encountered, Python stops the current loop, and the control flow is transferred to the following line of code immediately following the loop.
- The break statement is used to stop the loop immediately.
- When a break statement is found, the loop stops and the program returns to its normal execution beyond the loop.

```
numbers = [1, 4, 7, 8, 15, 20, 35, 45, 55]
for i in numbers:
    if i > 15:
        # break the loop
        break
    else:
        print(i)
```

Continue Statement in for loop

- The continue statement skips the current iteration of a loop and immediately jumps to the next iteration.
- Use the `continue` statement when you want to jump to the next iteration of the loop immediately. In simple terms, when the interpreter found the `continue` statement inside the loop, it skips the remaining code and moves to the next iteration.
- The `continue` statement skips a block of code in the loop for the current iteration only. It doesn't terminate the loop but continues in the next iteration ignoring the specified block of code.

```
name = "mariya mennen"
count = 0
for char in name:
    if char != 'm':
        continue
    else:
        count = count + 1

print('Total number of m is:', count)
```

Pass Statement in for loop

- The `pass` statement is a null statement, i.e., nothing happens when the statement is executed. Primarily it is used in empty functions or classes. When the interpreter finds a `pass` statement in the program, it returns no operation.

- Sometimes there is a situation in programming where we need to define a syntactically empty block. We can define that block with the pass keyword.

```
num = [1, 4, 5, 3, 7, 8]
for i in num:
    # calculate multiplication in future if required
    Pass
```

Reverse for loop

We can use the built-in function `reversed()` with `for` loop to change the order of elements, and this is the simplest way to perform a reverse looping.

```
# Reversed numbers using reversed() function
list1 = [10, 20, 30, 40]
for num in reversed(list1):
    print(num)
```

Reverse for loop using range()

We can use the built-in function `range()` with the `for` loop to reverse the elements' order.
The `range()` generates the integer numbers between the given start integer to the stop integer.

```
print("Reverse numbers using for loop")
num = 5
# start = 5
# stop = -1
# step = -1
for num in (range(num, -1, -1)):
    print(num)
```

Nested for loops

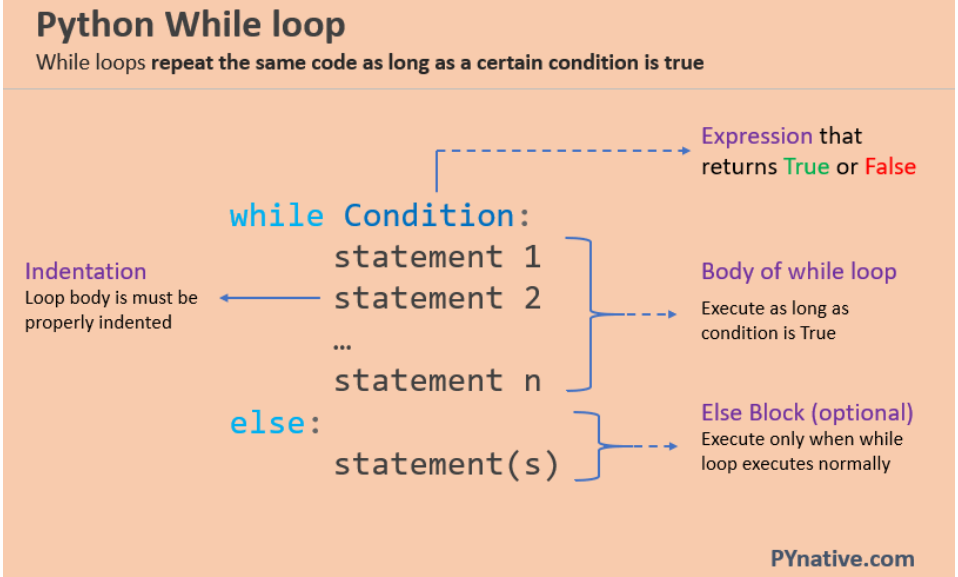
A nested loop has one loop inside of another. It is mainly used with two-dimensional arrays. For example, printing numbers or star patterns.

```
# outer for loop
for element in sequence
   # inner for loop
   for element in sequence:
       body of inner for loop
   body of outer for loop
other statements
```

Python While Loop

In Python, The while loop statement repeatedly executes a code block while a particular condition is true.

```
count = 1
# condition: Run loop till count is less than 3
while count < 3:
    print(count)
    count = count + 1
```

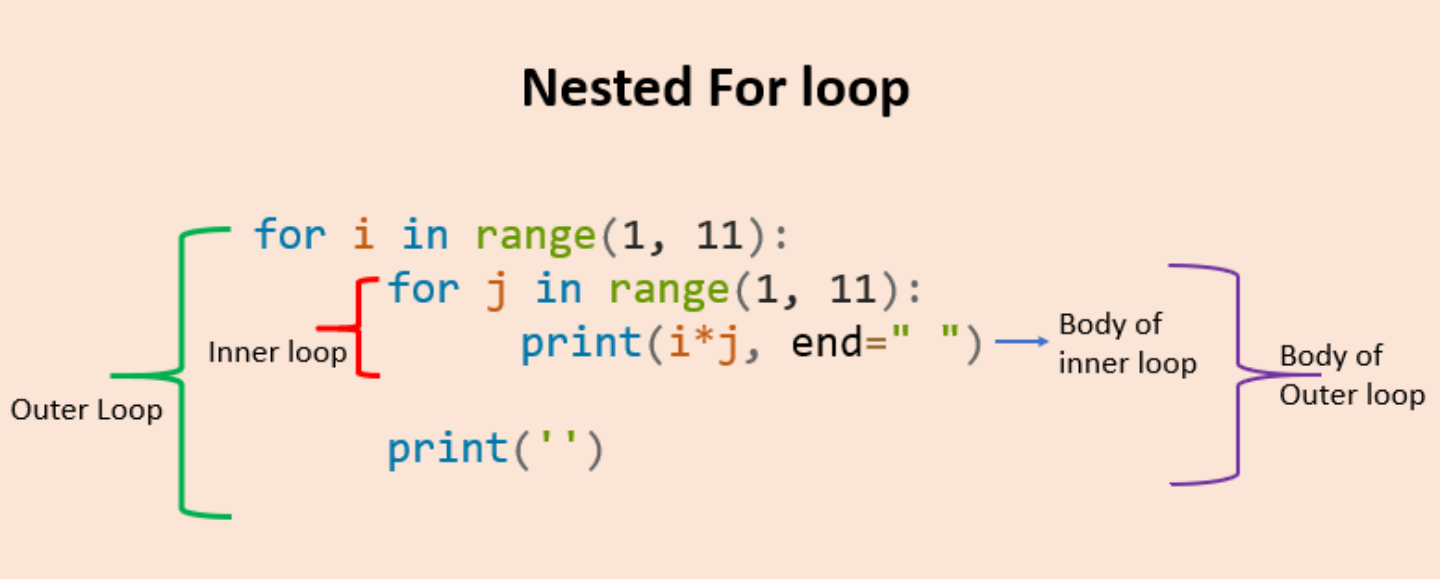


- The while statement checks the condition. The condition must return a boolean value. Either True or False.
- Next, If the condition evaluates to true, the while statement executes the statements present inside its block.
- The while statement continues checking the condition in each iteration and keeps executing its block until the condition becomes false.

Nested Loops in Python

- A nested loop is a loop inside the body of the outer loop. The inner or outer loop can be any type, such as a while loop or for loop. For example, the outer `for` loop can contain a `while` loop and vice versa.
- The outer loop can contain more than one inner loop. There is no limitation on the chaining of loops.
- In the nested loop, the number of iterations will be equal to the number of iterations in the outer loop multiplied by the iterations in the inner loop.
- In each iteration of the outer loop inner loop execute all its iteration. **For each iteration of an outer loop the inner loop re-start and completes its execution** before the outer loop can continue to its next iteration.
- Nested loops are typically used for working with multidimensional data structures, such as printing two-dimensional arrays, iterating a list that contains a nested list.

```
# outer for loop
for element in sequence
   # inner for loop
   for element in sequence:
       body of inner for loop
   body of outer for loop
```



Python Numbers

- Python offers a wide variety of inbuilt data types to handle different requirements. Each data type comes with built-in methods for performing different operations.
- The numeric data in Python is generally in three formats namely Integer, Float, and Complex.

Integer Number

Int or Integer data type is used for numeric data of any length without a decimal point. Integer data can be either positive or negative.

- Directly assigning an integer value to a variable
- Using a `int()` class.

```
roll_no = 33
# display roll no
print("Roll number is:", roll_no)  # 33
print(type(roll_no))  # class 'int'

# create integer using int() class
id = int(25)
print(id)  # 25
print(type(id))  # class 'int'
```

Float Number

- Float or Floating point data type is used for numeric data with one or more decimals. It can be either positive or negative.
- We can create a `Integer` or `Float` variable by assigning values. We can get the type of a variable by using the type function.

```
#data types
x=-9999
y=99.99
print(type(x))
print(type(y))
```

Complex Numbers

A Python complex number is one having a real and imaginary parts. It is generally represented as:-

```
c==c.real+c.imaginary*1j
```

Hexadecimal and Octal Numbers

We can represent a number in Octal form by preceding with a `0` and in hexadecimal form by preceding with a `0x`. We can convert a number from integer to octal by using the `oct()` method and into hexadecimal form by using the `hex()` method.

```
i = 10
print("in octal form::", oct(i))
print("in hexa decimal form ::", hex(i))
```

Implicit Type Conversion

- In the case of two compatible data types, Python performs the type conversion implicitly.

  Compatible data types are in general two numeric data types like int and float.

- It will convert the smaller data type to larger one to prevent any data loss. For eg.; int data will be converted to float.

```
x = 99
y = 1.111
#data in int and float
print(type(x))
print(type(y))
#addition
z = x+y
#type after addition
print(type(z))
```

Explicit Type Conversion

We can convert data in one type to another explicitly using `int(), float(), complex()` etc; This is called typecasting.

```
#concatenating int and str types
i = 99
s = "apple"
print(s+str(i))
```

Python Lists

The following are the **properties of a list**.

- **Mutable:** The elements of the list can be modified. We can add or remove items to the list after it has been created.
- **Ordered:** The items in the lists are ordered. Each item has a unique index value. The new items will be added to the end of the list.
- **Heterogenous**: The list can contain different kinds of elements i.e; they can contain elements of string, integer, boolean, or any type.
- **Duplicates:** The list can contain duplicates i.e., lists can have two items with the same values.

Why use a list?

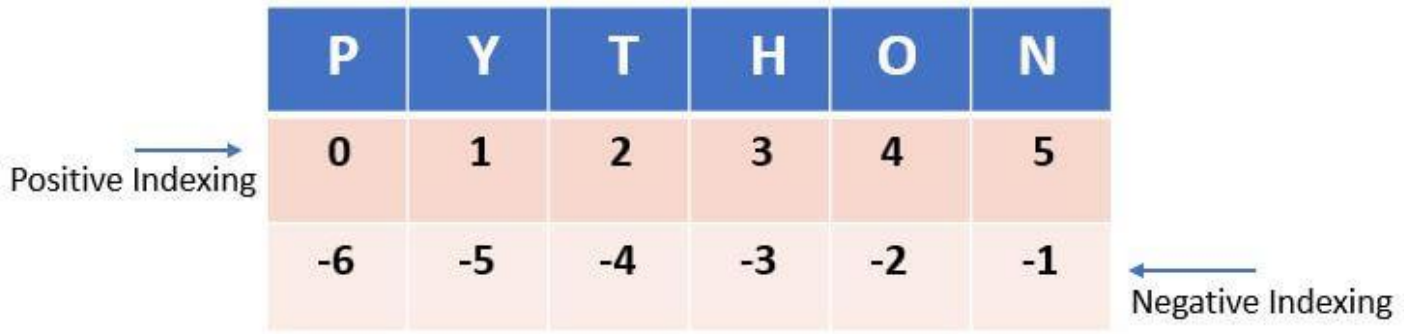
- The list data structure is very flexible It has many unique inbuilt functionalities like `pop()`, `append()`, etc which makes it easier, where the data keeps changing.
- Also, the list can contain duplicate elements i.e two or more items can have the same values.
- Lists are Heterogeneous i.e, different kinds of objects/elements can be added
- As Lists are mutable it is used in applications where the values of the items change frequently.

```
# Using list constructor
my_list1 = list((1, 2, 3))
print(my_list1)
# Output [1, 2, 3]
# Using square brackets[]
my_list2 = [1, 2, 3]
print(my_list2)
# Output [1, 2, 3]
# with heterogeneous items
my_list3 = [1.0, 'Jessa', 3]
print(my_list3)
# Output [1.0, 'Jessa', 3]
# empty list using list()
my_list4 = list()
print(my_list4)
```

```
# Output []
# empty list using []
my_list5 = []
print(my_list4)
# Output []
```

Indexing

The list elements can be accessed using the “indexing” technique. Lists are ordered collections with unique indexes for each item. We can access the items in the list using this index number.



List Slicing

Slicing a list implies, accessing a range of elements in a list. For example, if we want to get the elements in the position from 3 to 7, we can use the slicing method. We can even modify the values in a range by using this slicing technique.

```
listname[start_index : end_index : step]
```

Iterating a List

The objects in the list can be iterated over one by one, by using a for a loop.

```
my_list = [5, 8, 'Tom', 7.50, 'Emma']
# iterate a list
for item in my_list:
    print(item)
```

append()

The append() method will accept only one parameter and add it at the end of the list.

```
my_list = list([5, 8, 'Tom', 7.50])
# Using append()
my_list.append('Emma')
print(my_list)
# Output [5, 8, 'Tom', 7.5, 'Emma']
# append the nested list at the end
my_list.append([25, 50, 75])
print(my_list)
# Output [5, 8, 'Tom', 7.5, 'Emma', [25, 50, 75]]
```

insert()

Use the `insert()` method to add the object/item at the specified position in the list. The insert method accepts two parameters position and object.

```
insert(index, object)
```

```
my_list = list([5, 8, 'Tom', 7.50])
# Using insert()
# insert 25 at position 2
my_list.insert(2, 25)
print(my_list)
# Output [5, 8, 25, 'Tom', 7.5]
# insert nested list at at position 3
my_list.insert(3, [25, 50, 75])
print(my_list)
# Output [5, 8, 25, [25, 50, 75], 'Tom', 7.5]
```

extend()

The extend method will accept the list of elements and add them at the end of the list. We can even add another list by using this method.

```
my_list = list([5, 8, 'Tom', 7.50])
# Using extend()
my_list.extend([25, 75, 100])
print(my_list)
# Output [5, 8, 'Tom', 7.5, 25, 75, 100]
```

remove()

Use the `remove()` method to remove the first occurrence of the item from the list.

```
my_list = list([2, 4, 6, 8, 10, 12])
# remove item 6
my_list.remove(6)
# remove item 8
my_list.remove(8)
print(my_list)
# Output [2, 4, 10, 12]
```

pop()

Use the `pop()` method to remove the item at the given index. The `pop()` method removes and returns the item present at the given index.

```
my_list = list([2, 4, 6, 8, 10, 12])
# remove item present at index 2
my_list.pop(2)
print(my_list)
# Output [2, 4, 8, 10, 12]
# remove item without passing index number
my_list.pop()
print(my_list)
# Output [2, 4, 8, 10]
```

del()

Use del keyword along with list slicing to remove the range of items

```
my_list = list([2, 4, 6, 8, 10, 12])
# remove range of items
# remove item from index 2 to 5
del my_list[2:5]
print(my_list)
# Output [2, 4, 12]
# remove all items starting from index 3
my_list = list([2, 4, 6, 8, 10, 12])
del my_list[3:]
print(my_list)
# Output [2, 4, 6]
```

clear()

Use the list' `clear()` method to remove all items from the list. The `clear()` method truncates the list.

```
my_list = list([2, 4, 6, 8, 10, 12])
# clear list
my_list.clear()
print(my_list)
# Output []
# Delete entire list
del my_list
```

index()

The `index()` function will accept the value of the element as a parameter and returns the first occurrence of the element or returns `ValueError` if the element does not exist.

```
my_list = list([2, 4, 6, 8, 10, 12])
print(my_list.index(8))
# Output 3
# returns error since the element does not exist in the list.
# my_list.index(100)
```

Concatenation of two lists

The concatenation of two lists means merging of two lists. There are two ways to do that.

- Using the `+` operator.
- Using the `extend()` method. The `extend()` method appends the new list's items at the end of the calling list.

```
my_list1 = [1, 2, 3]
my_list2 = [4, 5, 6]
# Using + operator
my_list3 = my_list1 + my_list2
print(my_list3)
# Output [1, 2, 3, 4, 5, 6]
# Using extend() method
my_list1.extend(my_list2)
print(my_list1)
# Output [1, 2, 3, 4, 5, 6]
```

copy()

The copy method can be used to create a copy of a list. This will create a new list and any changes made in the original list will not reflect in the new list. This is **shallow copying**.

```
my_list1 = [1, 2, 3]
# Using copy() method
new_list = my_list1.copy()
# printing the new list
print(new_list)
# Output [1, 2, 3]
# making changes in the original list
my_list1.append(4)
# print both copies
print(my_list1)
# result [1, 2, 3, 4]
print(new_list)
# result [1, 2, 3]
```

sort()

The sort function sorts the elements in the list in ascending order.

```
mylist = [3,2,1]
mylist.sort()
print(mylist)
```

reverse()

The reverse function is used to reverse the elements in the list.

```
mylist = [3, 4, 5, 6, 1]
mylist.reverse()
print(mylist)
```

max() & min()

The max function returns the maximum value in the list while the min function returns the minimum value in the list.

```
mylist = [3, 4, 5, 6, 1]
print(max(mylist)) #returns the maximum number in the list.
print(min(mylist)) #returns the minimum number in the list.
```

Tuples in Python

Tuples are ordered collections of heterogeneous data that are unchangeable. Heterogeneous means tuple can store variables of all types.
Tuple has the following characteristics

- **Ordered**: Tuples are part of sequence data types, which means they hold the order of the data insertion. It maintains the index value for each item.
- **Unchangeable**: Tuples are unchangeable, which means that we cannot add or delete items to the tuple after creation.
- **Heterogeneous**: Tuples are a sequence of data of different data types (like integer, float, list, string, etc;) and can be accessed through indexing and slicing.
- **Contains Duplicates**: Tuples can contain duplicates, which means they can have items with the same value.

```
# create a tuple using ()
# number tuple
number_tuple = (10, 20, 25.75)
print(number_tuple)
# Output (10, 20, 25.75)
# string tuple
string_tuple = ('Jessa', 'Emma', 'Kelly')
print(string_tuple)
# Output ('Jessa', 'Emma', 'Kelly')
# mixed type tuple
sample_tuple = ('Jessa', 30, 45.75, [25, 78])
print(sample_tuple)
# Output ('Jessa', 30, 45.75, [25, 78])
# create a tuple using tuple() constructor
sample_tuple2 = tuple(('Jessa', 30, 45.75, [23, 78]))
print(sample_tuple2)
# Output ('Jessa', 30, 45.75, [23, 78])
```

len()

We can find the length of the tuple using the `len()` function. This will return the number of items in the tuple.

```
tuple1 = ('P', 'Y', 'T', 'H', 'O', 'N')
# length of a tuple
print(len(tuple1))
# Output 6
```

Iterating a Tuple

We can iterate a tuple using a for loop Let us see this with an example.

```
# create a tuple
sample_tuple = tuple((1, 2, 3, "Hello", [4, 8, 16]))
# iterate a tuple
for item in sample_tuple:
    print(item)
```

Slicing a tuple

We can even specify a range of items to be accessed from a tuple using the technique called 'Slicing.' The operator used is `':'`.

```
tuple1 = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
# slice a tuple with start and end index number
print(tuple1[1:5])
# Output (1, 2, 3, 4)
```

sum()

We can also use the Python built-in function `sum` to concatenate two tuples. But the sum function of two iterables like tuples always needs to start with Empty Tuple.

```
tuple1 = (1, 2, 3, 4, 5)
tuple2 = (3, 4, 5, 6, 7)
# using sum function
tuple3 = sum((tuple1, tuple2), ())
print(tuple3)
# Output (1, 2, 3, 4, 5, 3, 4, 5, 6, 7)
```

Sets in Python

In Python, a **Set is an unordered collection of data items that are unique**. In other words, Python Set is a collection of elements (Or objects) that contains no duplicate elements.

Characteristics of a Set

A set is a built-in data structure in Python with the following three characteristics.

- **Unordered:** The items in the set are unordered, unlike lists, i.e., it will not maintain the order in which the items are inserted. The items will be in a different order each time when we access the Set object. There will not be any index value assigned to each item in the set.
- **Unchangeable:** Set items must be immutable**.** We cannot change the set items, i.e., We cannot modify the items' value. But we can add or remove items to the Set. A set itself may be modified, but the elements contained in the set must be of an immutable type.
- **Unique:** There cannot be two items with the same value in the set.

```
# create a set using {}
# set of mixed types intger, string, and floats
sample_set = {'Mark', 'Jessa', 25, 75.25}
print(sample_set)
# Output {25, 'Mark', 75.25, 'Jessa'}
# create a set using set constructor
# set of strings
book_set = set(("Harry Potter", "Angels and Demons", "Atlas Shrugged"))
print(book_set)
# output {'Harry Potter', 'Atlas Shrugged', 'Angels and Demons'}
print(type(book_set))
# Output class 'set'
```

Dictionaries in Python

- **Dictionaries are ordered collections of unique values stored in (Key-Value) pairs.**
- In Python version **3.7** and onwards, dictionaries are ordered. In Python **3.6** and earlier, dictionaries are **unordered**.
- Python dictionary represents a mapping between a key and a value. In simple terms, a Python dictionary can store pairs of keys and values. Each key is linked to a specific value. Once stored in a dictionary, you can later obtain the value using just the key.

Characteristics of dictionaries

- **Unordered** (In Python 3.6 and lower version): The items in dictionaries are stored without any index value, which is typically a range of numbers. They are stored as Key-Value pairs, and the keys are their index, which will not be in any sequence.
- **Ordered** (In Python 3.7 and higher version): dictionaries are ordered, which means that the items have a defined order, and that order will not change. A simple Hash Table consists of key-value pair arranged in pseudo-random order based on the calculations from Hash Function.
- **Unique:** As mentioned above, each value has a Key; the Keys in Dictionaries should be unique. If we store any value with a Key that already exists, then the most recent value will replace the old value.
- **Mutable:** The dictionaries are changeable collections, which implies that we can add or remove items after the creation.

```
# create a dictionary using {}
person = {"name": "Jessa", "country": "USA", "telephone": 1178}
print(person)
# output {'name': 'Jessa', 'country': 'USA', 'telephone': 1178}
# create a dictionary using dict()
person = dict({"name": "Jessa", "country": "USA", "telephone": 1178})
print(person)
# output {'name': 'Jessa', 'country': 'USA', 'telephone': 1178}
# create a dictionary from sequence having each item as a pair
person = dict([("name", "Mark"), ("country", "USA"), ("telephone", 1178)])
```

```
print(person)
# create dictionary with mixed keys keys
# first key is string and second is an integer
sample_dict = {"name": "Jessa", 10: "Mobile"}
print(sample_dict)
# output {'name': 'Jessa', 10: 'Mobile'}
# create dictionary with value as a list
person = {"name": "Jessa", "telephones": [1178, 2563, 4569]}
print(person)
# output {'name': 'Jessa', 'telephones': [1178, 2563, 4569]}
```

Get all keys and values

Method	Description
`keys()`	Returns the list of all keys present in the dictionary.
`values()`	Returns the list of all values present in the dictionary
`items()`	Returns all the items present in the dictionary. Each item will be inside a tuple as a key-value pair.

```
person = {"name": "Jessa", "country": "USA", "telephone": 1178}
# Get all keys
print(person.keys())
# output dict_keys(['name', 'country', 'telephone'])
print(type(person.keys()))
# Output class 'dict_keys'
# Get all values
print(person.values())
# output dict_values(['Jessa', 'USA', 1178])
print(type(person.values()))
# Output class 'dict_values'
# Get all key-value pair
print(person.items())
# output dict_items([('name', 'Jessa'), ('country', 'USA'), ('telephone', 1178)])
print(type(person.items()))
# Output class 'dict_items'
```

Python Functions

- In Python, the **function is a block of code defined with a name**. We use functions whenever we need to perform the same task multiple times without writing the same code again. It can take arguments and returns the value.
- Python has a DRY principle like other programming languages. DRY stands for Don't Repeat Yourself. Consider a scenario where we need to do some action/task many times. We can define that action only once using a function and call that function whenever required to do the same activity.
- Function improves efficiency and reduces errors because of the reusability of a code. Once we create a function, we can call it anywhere and anytime. The benefit of using a function is reusability and modularity.

Types of Functions

Python support two types of functions

- Built-in function
- User-defined function

Built-in function

The functions which are come along with Python itself are called a built-in function or **predefined function**. Some of them are listed below.

`range()`, `id()`, `type()`, `input()`, `eval()` etc.

User-defined function

Functions which are created by programmer explicitly according to the requirement are called a user-defined function.

Creating a Function

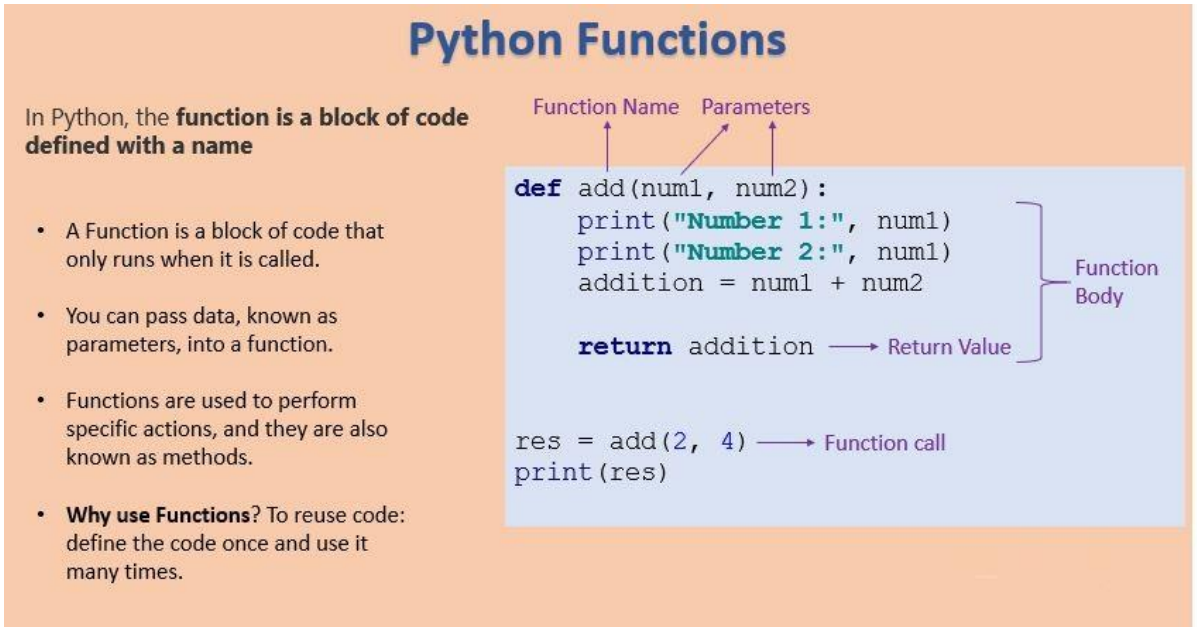
Use the following steps to to define a function in Python.

- Use the `def` keyword with the function name to define a function.
- Next, pass the number of parameters as per your requirement. (Optional).
- Next, define the function body with a **block of code**. This block of code is nothing but the action you wanted to perform.

In Python, no need to specify curly braces for the function body. The only **indentation** is essential to separate code blocks. Otherwise, you will get an error.

```
def function_name(parameter1, parameter2):
        # function body
        # write some action
return value
```

- `function_name`: Function name is the name of the function. We can give any name to function.
- `parameter`: Parameter is the value passed to the function. We can pass any number of parameters. Function body uses the parameter's value to perform an action
- `function_body`: The function body is a block of code that performs some task. This block of code is nothing but the action you wanted to accomplish.
- **return value**: Return value is the output of the function.



Creating a function without any parameters

Now, Let's the example of creating a simple function that prints a welcome message.

```
# function
def message():
    print("Welcome to PYnative")

# call function using its name
message()
```

Creating a function with parameters

Let's create a function that takes two parameters and displays their values.

```
# function
def course_func(name, course_name):
    print("Hello", name, "Welcome to PYnative")
    print("Your course name is", course_name)

# call function
course_func('John', 'Python')
```

Creating a function with parameters and return value

Functions can return a value. The return value is the output of the function. Use the return keyword to return value from a function.

```
# function
def calculator(a, b):
    add = a + b
    # return the addition
    return add

# call function
# take return value in variable
res = calculator(20, 5)

print("Addition :", res)
# Output Addition : 25
```

Calling a function

- Once we defined a function or finalized structure, we can call that function by using its name. We can also call that function from another function or program by importing it.
- To call a function, use the name of the function with the parenthesis, and if the function accepts parameters, then pass those parameters in the parenthesis.

```
# function
def even_odd(n):
    # check numne ris even or odd
    if n % 2 == 0:
        print('Even number')
    else:
        print('Odd Number')
# calling function by its name
even_odd(19)
# Output Odd Number
```

Return Value From a Function

In Python, to return value from the function, a `return` statement is used. It returns the value of the expression following the returns keyword.

```
def fun():
    statement-1
    statement-2
    statement-3
    .
    .
    return [expression]
```

Return Multiple Values

You can also return multiple values from a function. Use the return statement by separating each expression by a comma.

```
def arithmetic(num1, num2):
    add = num1 + num2
    sub = num1 - num2
    multiply = num1 * num2
    division = num1 / num2
    # return four values
    return add, sub, multiply, division
# read four return values in four variables
a, b, c, d = arithmetic(10, 2)
print("Addition: ", a)
print("Subtraction: ", b)
print("Multiplication: ", c)
print("Division: ", d)
```

Local Variable in function

A local variable is a variable declared inside the function that is not accessible from outside of the function. The scope of the local variable is limited to that function only where it is declared.
If we try to access the local variable from the outside of the function, we will get the error as NameError.

```
def function1():
    # local variable
    loc_var = 888
    print("Value is :", loc_var)
def function2():
    print("Value is :", loc_var)
function1()
function2()
```

Global Variable in function

A Global variable is a variable that declares outside of the function. The scope of a global variable is broad. It is accessible in all functions of the same module.

```
global_var = 999
def function1():
    print("Value in 1nd function :", global_var)
def function2():
    print("Value in 2nd function :", global_var)
function1()
function2()
```

Recursive Function

A recursive function is a function that calls itself, again and again.
Consider, calculating the factorial of a number is a repetitive activity, in that case, we can call a function again and again, which calculates factorial.

Python Exceptions and Errors

An exception is an **event that occurs during the execution of programs that disrupt the normal flow of execution** (e.g., KeyError Raised when a key is not found in a dictionary.) An exception is a Python object that represents an error.
In Python, an exception is an object derives from the **BaseException** class that contains information about an error event that occurred within a method. **Exception object contains**:

- Error type (exception name)
- The state of the program when the error occurred
- An error message describes the error event.

Exception are useful to indicate different types of possible failure condition.

For example, bellow are the few standard exceptions

- FileNotFoundException
- ImportError
- RuntimeError
- NameError
- TypeError

What are Errors?

On the other hand, An **error** is an action that is incorrect or inaccurate. For example, syntax error. Due to which the program fails to execute.

Syntax error
The syntax error occurs when we are not following the proper structure or syntax of the language. A syntax error is also known as a **parsing error**.
Common Python Syntax errors:

- Incorrect indentation
- Missing colon, comma, or brackets
- Putting keywords in the wrong place.

Logical errors (Exception)

Even if a statement or expression is syntactically correct, the error that occurs at the runtime is known as a **Logical error or Exception**. In other words, **Errors detected during execution are called exceptions**.

Common Python Logical errors:

- Indenting a block to the wrong level
- using the wrong variable name
- making a mistake in a boolean expression

Built-in Exceptions

Python automatically generates many exceptions and errors. Runtime exceptions, generally a result of programming errors, such as:

- Reading a file that is not present
- Trying to read data outside the available index of a list
- Dividing an integer value by zero

Exception	Description
`AssertionError`	Raised when an `assert` statement fails.
`AttributeError`	Raised when attribute assignment or reference fails.
`EOFError`	Raised when the `input()` function hits the end-of-file condition.
`FloatingPointError`	Raised when a floating-point operation fails.
`GeneratorExit`	Raise when a generator's close() method is called.
`ImportError`	Raised when the imported module is not found.
`IndexError`	Raised when the index of a sequence is out of range.
`KeyError`	Raised when a key is not found in a dictionary.
`KeyboardInterrupt`	Raised when the user hits the interrupt key (Ctrl+C or Delete)
`MemoryError`	Raised when an operation runs out of memory.
`NameError`	Raised when a variable is not found in the local or global scope.
`OSError`	Raised when system operation causes system related error.
`ReferenceError`	Raised when a weak reference proxy is used to access a garbage collected referent.

The try and except Block to Handling Exceptions

- When an exception occurs, Python stops the program execution and generates an exception message. It is highly recommended to handle exceptions. The doubtful code that may raise an exception is called risky code.
- To handle exceptions we need to use try and except block. Define risky code that can raise an exception inside the try block and corresponding handling code inside the except block.

```
try :
    # statements in try block
except :
    # executed when exception occured in try block
```

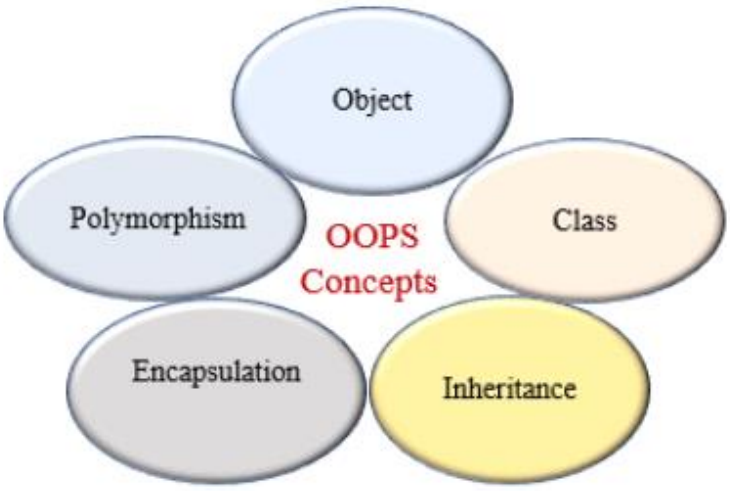
Using try with finally

- Python provides the finally block, which is used with the try block statement. **The finally block is used to write a block of code that must execute, whether the try block raises an error or not**.
- Mainly, the finally block is used to release the external resource. This block provides a guarantee of execution.

```
try:
    # block of code
    # this may throw an exception
finally:
    # block of code
    # this will always be executed
    # after the try and any except block
```

Object-Oriented Programming (OOP)

- **Object-oriented programming** (OOP) is a programming paradigm based on the concept of "**objects**". The object contains both data and code: Data in the form of properties (often known as attributes), and code, in the form of methods (actions object can perform).
- An object-oriented paradigm is to design the program using classes and objects. Python programming language supports different programming approaches like functional programming, modular programming.
- One of the popular approaches is object-oriented programming (OOP) to solve a programming problem is by creating objects.



An object has the following two characteristics:

- Attribute
- Behavior

For example, A Car is an object, as it has the following properties:

- name, price, color as attributes
- breaking, acceleration as behavior

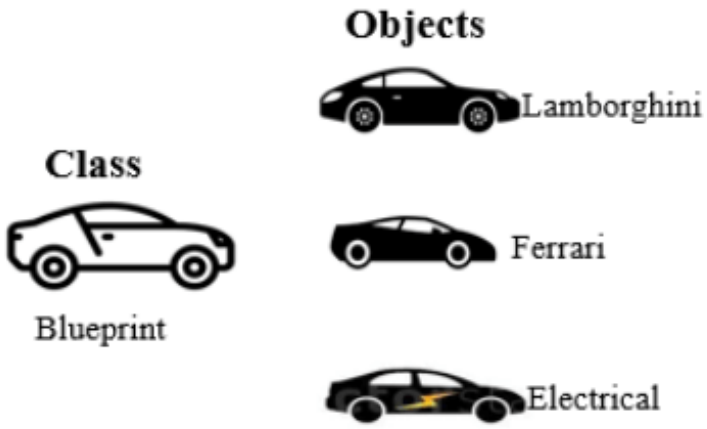
One important aspect of OOP in Python is to create **reusable code** using the concept of inheritance. This concept is also known as DRY (Don't Repeat Yourself).

Class and Objects

- In Python, everything is an object. A **class is a blueprint for the object**. To create an object we require a model or plan or blueprint which is nothing but class.

A class contains the properties (attribute) and action (behavior) of the object. Properties represent variables, and the methods represent actions. Hence class includes both variables and methods.

For example, you are creating a vehicle according to the Vehicle blueprint (template). The plan contains all dimensions and structure. Based on these descriptions, we can construct a car, truck, bus, or any vehicle. Here, a car, truck, bus are objects of Vehicle class.



Object is an instance of a class. The physical existence of a class is nothing but an object. In other words, the object is an entity that has a state and behavior. It may be any real-world object like the mouse, keyboard, laptop, etc.

Class Attributes and Methods

When we design a class, we use instance variables and class variables.
In Class, attributes can be defined into two parts:

- **Instance variables**: The instance variables are attributes attached to an instance of a class. We define instance variables in the constructor (the `__init__()` method of a class).
- **Class Variables:** A class variable is a variable that is declared inside of class, but outside of any instance method or `__init()__` method.

Inside a Class, we can define the following three types of methods.

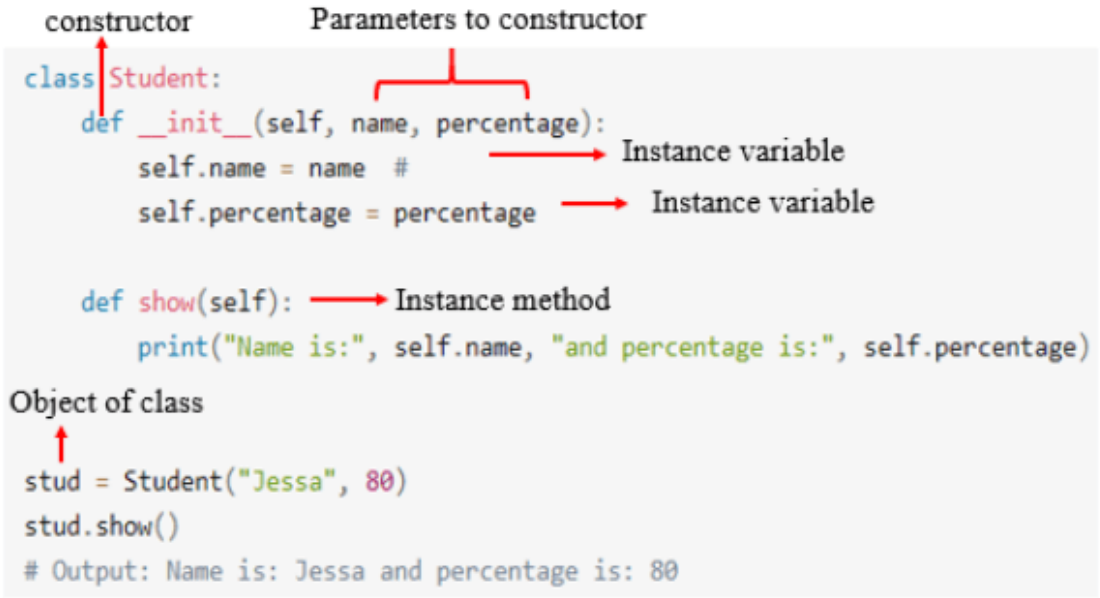
- **Instance method:** Used to access or modify the object attributes. If we use instance variables inside a method, such methods are called instance methods.

- **Class method**: Used to access or modify the class state. In method implementation, if we use only class variables, then such type of methods we should declare as a class method.
- **Static method**: It is a general utility method that performs a task in isolation. Inside this method, we don't use instance or class variable because this static method doesn't have access to the class attributes.

Creating Class and Objects

- In Python, Use the keyword `class` to define a Class. In the class definition, the first string is docstring which, is a brief description of the class.
- The docstring is not mandatory but recommended to use. We can get docstring using `__doc__` attribute. Use the following syntax to create a `class`.

```
class Employee:
    # class variables
    company_name = 'ABC Company'
    # constructor to initialize the object
    def __init__(self, name, salary):
        # instance variables
        self.name = name
        self.salary = salary
    # instance method
    def show(self):
        print('Employee:', self.name, self.salary, self.company_name)
# create first object
emp1 = Employee("Harry", 12000)
emp1.show()
# create second object
emp2 = Employee("Emma", 10000)
emp2.show()
```



Constructors in Python

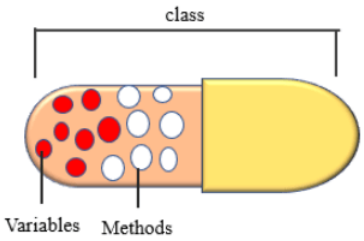
- In Python, a **constructor** is a special type of method used to initialize the object of a Class. The constructor will be executed automatically when the object is created. If we create three objects, the constructor is called three times and initialize each object.
- The main purpose of the constructor is to declare and initialize instance variables. It can take at least one argument that is self. The __init()__ method is called the constructor in Python. In other words, the name of the constructor should be __init__(self).

A constructor is optional, and if we do not provide any constructor, then Python provides the default constructor. Every class in Python has a constructor, but it's not required to define it.

Encapsulation in Python

- In Python, encapsulation is a method of wrapping data and functions into a single entity. For example, A class encapsulates all the data (methods and variables). Encapsulation means the internal representation of an object is generally hidden from outside of the object's definition.

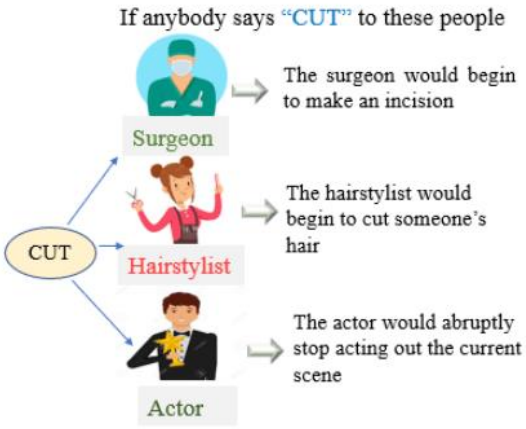
Encapsulation acts as a protective layer. We can restrict access to methods and variables from outside, and It can prevent the data from being modified by accidental or unauthorized modification. Encapsulation provides security by hiding the data from the outside world.




```
class Employee:
    def __init__(self, name, salary):
        # public member
        self.name = name
        # private member
        # not accessible outside of a class
        self.__salary = salary
    def show(self):
        print("Name is ", self.name, "and salary is", self.__salary)
emp = Employee("Jessa", 40000)
emp.show()
# access salary from outside of a class
print(emp.__salary)
```

Polymorphism in Python

- Polymorphism in OOP is the **ability of an object to take many forms**. In simple words, polymorphism allows us to perform the same action in many different ways.
- Polymorphism is taken from the Greek words Poly (many) and morphism (forms). Polymorphism defines the ability to take different forms.




```
class Circle:
    pi = 3.14
    def __init__(self, redius):
        self.radius = redius
    def calculate_area(self):
        print("Area of circle :", self.pi * self.radius * self.radius)
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def calculate_area(self):
        print("Area of Rectangle :", self.length * self.width)
# function
def area(shape):
    # call action
    shape.calculate_area()
# create object
cir = Circle(5)
rect = Rectangle(10, 5)
# call common function
area(cir)
area(rect)
```

Inheritance In Python

- In an Object-oriented programming language, inheritance is an important aspect. In Python, inheritance is the process of inheriting the properties of the parent class into a child class.
- The primary purpose of inheritance is the reusability of code. Using inheritance, we can use the existing class to create a new class instead of recreating it from scratch.

```
# Base class
class Vehicle:
    def __init__(self, name, color, price):
        self.name = name
        self.color = color
        self.price = price
    def info(self):
        print(self.name, self.color, self.price)
# Child class
class Car(Vehicle):
    def change_gear(self, no):
        print(self.name, 'change gear to number', no)
# Create object of Car
car = Car('BMW X1', 'Black', 35000)
car.info()
car.change_gear(5)
```

Python File Operation

- A file is a named location used for storing data. For example, main.py is a file that is always used to store Python code.
- Python provides various functions to perform different file operations, a process known as File Handling.

Opening Files in Python

In Python, we need to open a file first to perform any operations on it—we use the open() function to do so.

```
file1 = open("file1.txt")
```

Different Modes to Open a File in Python

Mode	Description
r	Open a file in reading mode (default)
w	Open a file in writing mode
x	Open a file for exclusive creation
a	Open a file in appending mode (adds content at the end of the file)
t	Open a file in text mode (default)
b	Open a file in binary mode
+	Open a file in both read and write mode

Reading Files in Python

After we open a file, we use the read() method to read its content.

```
# open a file in read mode
file1 = open("file1.txt")
# read the file content
read_content = file1.read()
print(read_content)
```

Writing to Files in Python

To write to a Python file, we need to open it in write mode using the w parameter.

```
# open the file2.txt in write mode
file2 = open('file2.txt', 'w')
# write contents to the file2.txt file
file2.write('Programming is Fun.\n')
file2.write('Programiz for beginners\n')
```

Closing Files in Python

When we are done performing operations on the file, we need to close the file properly. We use the close() function to close a file in Python.

```
# open a file
file1 = open("file1.txt", "r")
# read the file
read_content = file1.read()
print(read_content)
# close the file
file1.close()
```

Opening a Python File Using with...open

In Python, there is a better way to open a file using with...open.

```
with open("file1.txt", "r") as file1:
    read_content = file1.read()
    print(read_content)
```

Here, with...open automatically closes the file, so we don't have to use the close() function.

Python Directory and Files Management

- A directory is a collection of files and subdirectories. A directory inside a directory is known as a subdirectory.
- Python has the os module that provides us with many useful methods to work with directories (and files as well).

Get Current Directory in Python

We can get the present working directory using the getcwd() method of the os module. This method returns the current working directory in the form of a string.

```
import os
print(os.getcwd())
# Output: C:\Program Files\PyScripter
```

Changing Directory in Python

- In Python, we can change the current working directory by using the chdir() method.
- The new path that we want to change into must be supplied as a string to this method. And we can use both the forward-slash / or the backward-slash \ to separate the path elements.

```
import os
# change directory
os.chdir('C:\\Python33')
print(os.getcwd())
Output: C:\Python33
```

Making a New Directory in Python

- In Python, we can make a new directory using the mkdir() method.
- This method takes in the path of the new directory. If the full path is not specified, the new directory is created in the current working directory.

```
os.mkdir('test')
os.listdir()
['test']
```

Renaming a Directory or a File

The rename() method can rename a directory or a file. For renaming any directory or file, rename() takes in two basic arguments:

- the old name as the first argument
- the new name as the second argument.

```
import os
os.listdir()
['test']
# rename a directory
os.rename('test','new_one')
os.listdir()
['new_one']
```

Removing Directory or File in Python

In Python, we can use the remove() method or the rmdir() method to remove a file or directory.

```
import os
# delete "myfile.txt" file
os.remove("myfile.txt")
```

```
import os
# delete the empty directory "mydir"
os.rmdir("mydir")
```

```
import shutil
# delete "mydir" directory and all of its contents
shutil.rmtree("mydir")
```

Python CSV: Read and Write CSV Files

The CSV (Comma Separated Values) format is a common and straightforward way to store tabular data. To represent a CSV file, it should have the .csv file extension.

Working With CSV Files in Python

Python provides a dedicated csv module to work with csv files. The module includes various methods to perform different operations.

```
import csv
```

Read CSV Files with Python

The csv module provides the csv.reader() function to read a CSV file.

```
import csv
with open('people.csv', 'r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Write to CSV Files with Python

The csv module provides the csv.writer() function to write to a CSV file.

```
import csv
with open('protagonist.csv', 'w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(["SN", "Movie", "Protagonist"])
    writer.writerow([1, "Lord of the Rings", "Frodo Baggins"])
    writer.writerow([2, "Harry Potter", "Harry Potter"])
```

Python String

A string is a sequence of characters. Python treats anything inside quotes as a string. This includes letters, numbers, and symbols. Python has no character data type so single character is a string of length 1.

Python String capitalize()

The capitalize() method converts the first character of a string to an uppercase letter and all other alphabets to lowercase.

```
sentence = "i love PYTHON"
# converts first character to uppercase and others to lowercase
capitalized_string = sentence.capitalize()
print(capitalized_string)
# Output: I love python
```

Python String center()

The center() method returns a new centered string after padding it with the specified character.

```
sentence = "Python is awesome"
# returns the centered padded string of length 24
new_string = sentence.center(24, '*')
print(new_string)
# Output: ***Python is awesome****
```

Python String casefold()

The casefold() method converts all characters of the string into lowercase letters and returns a new string.

```
text = "pYtHon"
# convert all characters to lowercase
lowercased_string = text.casefold()
print(lowercased_string)
# Output: python
```

Python String count()

The count() method returns the number of occurrences of a substring in the given string.

```
message = 'python is popular programming language'
# number of occurrence of 'p'
print('Number of occurrence of p:', message.count('p'))
# Output: Number of occurrence of p: 4
```

Python String endswith()

The endswith() method returns True if a string ends with the specified suffix. If not, it returns False.

```
message = 'Python is fun'
# check if the message ends with fun
print(message.endswith('fun'))
# Output: True
```

Python String expandtabs()

The expandtabs() takes an integer tabsize argument. The default tabsize is 8.

```
str = 'xyz\t12345\tabc'
# no argument is passed
# default tabsize is 8
result = str.expandtabs()
print(result)
```

Python String encode()

The encode() method returns an encoded version of the given string.

```
title = 'Python Programming'
# change encoding to utf-8
print(title.encode())
# Output: b'Python Programming'
```

Python String find()

The find() method returns the index of first occurrence of the substring (if found). If not found, it returns -1.

```
message = 'Python is a fun programming language'
# check the index of 'fun'
print(message.find('fun'))
# Output: 12
```

Python String index()

The index() method returns the index of a substring inside the string (if found). If the substring is not found, it raises an exception.

```
text = 'Python is fun'
# find the index of is
result = text.index('is')
print(result)
# Output: 7
```

Python String isalnum()

The isalnum() method returns True if all characters in the string are alphanumeric (either alphabets or numbers). If not, it returns False.

```
# string contains either alphabet or number
name1 = "Python3"
print(name1.isalnum()) #True
# string contains whitespace
name2 = "Python 3"
print(name2.isalnum()) #False
```

Python String isalpha()

isalpha() doesn't take any parameters.

```
name = "Monica"
print(name.isalpha())
# contains whitespace
name = "Monica Geller"
print(name.isalpha())
# contains number
name = "Mo3nicaGell22er"
print(name.isalpha())
```

Python String isdecimal()

The isdecimal() doesn't take any parameters.

```
s = "28212"
print(s.isdecimal())
# contains alphabets
s = "32ladk3"
print(s.isdecimal())
# contains alphabets and spaces
s = "Mo3 nicaG el l22er"
print(s.isdecimal())
```

Python String isdigit()

The isdigit() method returns True if all characters in a string are digits. If not, it returns False.

```
str1 = '342'
print(str1.isdigit())
str2 = 'python'
print(str2.isdigit())
# Output: True
#         False
```

Python String islower()

The islower() method doesn't take any parameters.

```
s = 'this is good'
print(s.islower())
s = 'th!s is a1so g00d'
print(s.islower())
s = 'this is Not good'
print(s.islower())
```

Python String isnumeric()

The isnumeric() method checks if all the characters in the string are numeric.

```
pin = "523"
# checks if every character of pin is numeric
print(pin.isnumeric())
# Output: True
```

Python String isspace()

Characters that are used for spacing are called whitespace characters. For example: tabs, spaces, newline, etc.

```
s = '   \t'
print(s.isspace())
s = ' a '
print(s.isspace())
s = ''
print(s.isspace())
```

Python String isupper()

The isupper() method doesn't take any parameters.

```
# example string
string = "THIS IS GOOD!"
print(string.isupper());
# numbers in place of alphabets
string = "THIS IS ALSO G00D!"
```

```
print(string.isupper());
# lowercase string
string = "THIS IS not GOOD!"
print(string.isupper());
```

Python String join()

The string join() method returns a string by joining all the elements of an iterable (list, string, tuple), separated by the given separator.

```
text = ['Python', 'is', 'a', 'fun', 'programming', 'language']
# join elements of text with space
print(' '.join(text))
# Output: Python is a fun programming language
```

Python String ljust()

```
# example string
string = 'cat'
width = 5
# print left justified string
print(string.ljust(width))
```

Python String lower()

The lower() method converts all uppercase characters in a string into lowercase characters and returns it.

```
message = 'PYTHON IS FUN'
# convert message to lowercase
print(message.lower())
# Output: python is fun
```

Python String upper()

The upper() method converts all lowercase characters in a string into uppercase characters and returns it.

```
message = 'python is fun'
# convert message to uppercase
print(message.upper())
# Output: PYTHON IS FUN
```